

**UNIVERSIDADE FEDERAL DE VIÇOSA
CAMPUS DE FLORESTAL
INSTITUTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
CIÊNCIA DA COMPUTAÇÃO**

**Relatório - Trabalho Prático 02
Caminho de Dados do RISC-V**

CCF 252 - Organização de Computadores I

**VINÍCIUS ARRUDA MELO MIRANDA - 5930
Grupo 22**

**Florestal - MG
2026**

Sumário

1	Introdução	3
2	Desenvolvimento	3
2.1	Subconjunto de instruções	3
2.2	Arquivos e módulos do projeto	3
2.3	Unidade de controle	4
2.4	Controle da ALU	6
2.5	ALU	7
2.6	Gerador de imediatos	8
2.7	Banco de registradores	8
2.8	Memória de dados	9
2.9	Memória de instruções	10
2.10	Caminho de dados	10
3	Simulação	12
3.1	Comando utilizado	12
3.2	Código de teste em ASM	12
3.3	Código em linguagem de máquina	12
3.4	Resultados esperados da simulação	13
4	Quartus e FPGA	13
4.1	Configuração do projeto no Quartus	13
4.2	Módulo principal da FPGA	14
4.3	Exibição do PC e dos registradores	15
4.4	Comandos na placa	16
5	Resultados	16
5.1	Resultado da simulação	16
5.2	Resultado no Quartus	17
5.3	Resultado na FPGA	17
6	Código no GitHub	17

7	Código Github	17
8	Considerações finais	17

1 Introdução

Neste trabalho prático foi implementada uma versão simplificada do caminho de dados do RISC-V utilizando Verilog/SystemVerilog. O objetivo principal foi permitir a execução de um subconjunto específico de instruções, verificando o funcionamento integrado entre contador de programa, memória de instruções, banco de registradores, unidade de controle, ALU, memória de dados e multiplexadores.

O projeto foi dividido em duas partes. Na primeira parte, o processador foi simulado no computador utilizando um testbench com clock automático, permitindo observar os registradores e a memória de dados pelo terminal. Na segunda parte, o caminho de dados foi adaptado para síntese no Quartus Prime e execução na placa DE2-115, com exibição do contador de programa e dos registradores em displays de sete segmentos e LEDs.

2 Desenvolvimento

2.1 Subconjunto de instruções

O grupo 22 ficou responsável pela implementação das seguintes instruções RISC-V:

- **lb** - carrega um byte da memória de dados para um registrador, com extensão de sinal;
- **sb** - armazena um byte de um registrador na memória de dados;
- **sub** - realiza subtração entre dois registradores;
- **and** - realiza operação lógica AND entre dois registradores;
- **ori** - realiza operação lógica OR entre registrador e imediato;
- **srl** - realiza deslocamento lógico à direita;
- **beq** - realiza desvio condicional quando dois registradores possuem valores iguais.

Essas instruções exigem suporte a formatos diferentes do RISC-V: tipo R para *sub*, *and* e *srl*; tipo I para *lb* e *ori*; tipo S para *sb*; e tipo B para *beq*.

2.2 Arquivos e módulos do projeto

A implementação foi organizada em módulos, seguindo a estrutura do caminho de dados utilizado em aula. A Tabela 1 resume a função de cada módulo.

Tabela 1: Principais módulos utilizados no projeto.

Módulo	Função no projeto
alu	Executa operações aritméticas e lógicas, como soma para cálculo de endereço, subtração, AND, OR e deslocamento lógico à direita.
Alu_Control	Define a operação da ALU a partir do <code>aluOp</code> , <code>funct3</code> e <code>funct7</code> .
Control	Gera os sinais de controle do processador a partir do opcode da instrução.
immGen	Extraí e estende os imediatos das instruções dos tipos I, S e B.
Reg_mem	Implementa o banco de 32 registradores do RISC-V, mantendo o registrador x0 sempre igual a zero.
data_mem	Implementa a memória de dados utilizada pelas instruções <i>lb</i> e <i>sb</i> .
Instruction_mem	Armazena as instruções manualmente no código, sem uso de <code>\$readmemb</code> ou <code>\$readmemh</code> .
pc	Armazena o valor atual do Program Counter.
data_pathR5	Integra todos os módulos do caminho de dados.
button_pulse	Gera pulsos a partir dos botões físicos da FPGA, reduzindo múltiplas leituras indesejadas.
seg7	Converte valores binários em sinais para displays de sete segmentos.
DE2_115	Módulo principal da implementação física na placa, conectando botões, switches, LEDs e displays ao processador.

2.3 Unidade de controle

A unidade de controle recebe o opcode da instrução e gera os sinais necessários para definir o comportamento do caminho de dados. No caso do grupo 22, foram tratados os opcodes das instruções de carga, armazenamento, tipo R, tipo I e desvio condicional.

Listing 1: Trecho principal da unidade de controle.

```

1 module Control(opcode, branch, memRead, memToReg, aluOp,
2               memWrite, aluSrc, regWrite);
3     input [6:0] opcode;
4     output reg branch, memRead, memToReg, memWrite, aluSrc, regWrite;
```

```

5      output reg [1:0] aluOp;
6
7      always @(*) begin
8          branch = 1'b0;
9          memRead = 1'b0;
10         memToReg = 1'b0;
11         aluOp = 2'b00;
12         memWrite = 1'b0;
13         aluSrc = 1'b0;
14         regWrite = 1'b0;
15
16         case (opcode)
17             7'b0000011: begin // lb
18                 memRead = 1'b1;
19                 memToReg = 1'b1;
20                 aluSrc = 1'b1;
21                 regWrite = 1'b1;
22             end
23
24             7'b0100011: begin // sb
25                 memWrite = 1'b1;
26                 aluSrc = 1'b1;
27             end
28
29             7'b0110011: begin // sub, and, srl
30                 aluOp = 2'b10;
31                 regWrite = 1'b1;
32             end
33
34             7'b0010011: begin // ori
35                 aluOp = 2'b11;
36                 aluSrc = 1'b1;
37                 regWrite = 1'b1;
38             end
39
40             7'b1100011: begin // beq
41                 branch = 1'b1;
42                 aluOp = 2'b01;
43             end
44         endcase
45     end

```

Para *lb*, a ALU calcula o endereço, a memória é lida e o valor retorna para o registrador. Para *sb*, a memória é escrita, mas nenhum registrador recebe valor. Para instruções tipo R, a escrita no banco de registradores é habilitada. Para *beq*, o sinal branch é ativado e a ALU realiza uma subtração para verificar se os dois operandos são iguais.

2.4 Controle da ALU

O módulo `Alu_Control` traduz o sinal `aluOp`, gerado pela unidade de controle, junto com os campos `funct3` e `funct7`, para selecionar a operação correta da ALU.

Listing 2: Trecho principal do controle da ALU.

```

1 module Alu_Control(funct7, funct3, aluOp, aluCtrl);
2     input [6:0] funct7;
3     input [2:0] funct3;
4     input [1:0] aluOp;
5     output reg [3:0] aluCtrl;
6
7     always @(*) begin
8         case (aluOp)
9             2'b00: aluCtrl = 4'b0000; // endereco lb/sb
10            2'b01: aluCtrl = 4'b0001; // comparacao do beq
11
12            2'b10: begin
13                case (funct3)
14                    3'b000: aluCtrl = (funct7 == 7'b0100000)
15                        ? 4'b0001 : 4'b0000; // sub
16                    3'b111: aluCtrl = 4'b0010; // and
17                    3'b101: aluCtrl = 4'b0100; // srl
18                    default: aluCtrl = 4'b0000;
19                endcase
20            end
21
22            2'b11: begin
23                case (funct3)
24                    3'b110: aluCtrl = 4'b0011; // ori
25                    default: aluCtrl = 4'b0000;
26                endcase
27            end
28        endcase
29    end

```

```

29         default: aluCtrl = 4'b0000;
30     endcase
31 end
32 endmodule

```

A separação entre `Control` e `Alu_Control` torna o projeto mais organizado, pois a unidade principal identifica o tipo geral de instrução, enquanto o controle da ALU escolhe a operação específica.

2.5 ALU

A ALU implementada suporta soma, subtração, AND, OR e deslocamento lógico à direita. A soma é utilizada principalmente para calcular o endereço de memória em *lb* e *sb*. A subtração é usada tanto pela instrução *sub* quanto pela comparação da instrução *beq*.

Listing 3: Operações implementadas na ALU.

```

1 module alu(inputA, inputB, aluCtrl, aluResult, zero);
2     input [31:0] inputA, inputB;
3     input [3:0] aluCtrl;
4     output reg [31:0] aluResult;
5     output zero;
6
7     always @(*) begin
8         case (aluCtrl)
9             4'b0000: aluResult = inputA + inputB;
10            4'b0001: aluResult = inputA - inputB;
11            4'b0010: aluResult = inputA & inputB;
12            4'b0011: aluResult = inputA | inputB;
13            4'b0100: aluResult = inputA >> inputB[4:0];
14            default: aluResult = 32'b0;
15        endcase
16    end
17
18    assign zero = (aluResult == 32'b0);
19 endmodule

```

O sinal `zero` é utilizado no desvio condicional. Quando a subtração entre os dois registradores do *beq* resulta em zero, significa que os valores são iguais e o branch deve ser tomado.

2.6 Gerador de immediatos

Como as instruções possuem formatos diferentes, o imediato precisa ser extraído de posições diferentes da instrução. O módulo `immGen` trata os formatos I, S e B.

Listing 4: Extração dos immediatos no módulo `immGen`.

```
1 module immGen(instruction, immediate);
2     input [31:0] instruction;
3     output reg [31:0] immediate;
4
5     wire [6:0] opcode;
6     assign opcode = instruction[6:0];
7
8     always @(*) begin
9         case (opcode)
10             7'b0000011, // lb
11             7'b0010011: // ori
12                 immediate = {{20{instruction[31]}}, instruction[31:20]};
13
14             7'b0100011: // sb
15                 immediate = {{20{instruction[31]}},
16                             instruction[31:25], instruction[11:7]};
17
18             7'b1100011: // beq
19                 immediate = {{19{instruction[31]}}, instruction[31],
20                             instruction[7], instruction[30:25],
21                             instruction[11:8], 1'b0};
22
23             default:
24                 immediate = 32'b0;
25         endcase
26     end
27 endmodule
```

No caso do *beq*, o último bit do imediato é sempre zero, pois o endereço de destino precisa estar alinhado. Por isso, o imediato é montado com `1'b0` no final.

2.7 Banco de registradores

O banco de registradores possui 32 posições de 32 bits, seguindo a arquitetura RISC-V. O registrador `x0` é mantido sempre com valor zero, mesmo que alguma instrução tente escrever nele.

Listing 5: Escrita no banco de registradores.

```
1 always @(posedge clock or posedge reset) begin
2     if (reset) begin
3         for (i = 0; i < 32; i = i + 1)
4             registers[i] <= 32'b0;
5     end
6     else begin
7         if (regWrite && writeReg != 5'b00000)
8             registers[writeReg] <= writeData;
9
10            registers[0] <= 32'b0;
11    end
12 end
13
14 assign readData1 = (readReg1 == 5'b00000) ? 32'b0 : registers[
15     readReg1];
16 assign readData2 = (readReg2 == 5'b00000) ? 32'b0 : registers[
17     readReg2];
```

Na versão para FPGA foi adicionado um sinal de depuração, permitindo selecionar e exibir o valor de um registrador específico nos displays e LEDs.

2.8 Memória de dados

A memória de dados foi implementada como uma memória endereçada por byte, pois o grupo utiliza as instruções *lb* e *sb*. A instrução *sb* grava apenas os 8 bits menos significativos do registrador, e a instrução *lb* carrega um byte da memória com extensão de sinal.

Listing 6: Leitura e escrita de byte na memória de dados.

```
1 always @(posedge clock or posedge reset) begin
2     if (reset) begin
3         for (i = 0; i < 128; i = i + 1)
4             memory[i] <= 8'b0;
5     end
6     else if (memWrite) begin
7         memory[address[6:0]] <= writeData[7:0];
8     end
9 end
10
11 always @(*) begin
12     if (memRead)
```

```

13         readData = {{24{memory[address[6:0]][7]}},
14                     memory[address[6:0]]};
15     else
16         readData = 32'b0;
17 end

```

Essa implementação permite testar o comportamento específico das instruções de byte. No programa utilizado, o valor 7 é salvo na posição zero da memória por *sb* e depois carregado para o registrador x6 por *lb*.

2.9 Memória de instruções

Na versão final, as instruções foram inseridas diretamente no código Verilog. Isso foi necessário para deixar a memória de instruções adequada à síntese no Quartus, sem depender de leitura externa por arquivo.

Listing 7: Instruções inseridas manualmente na memória de instruções.

```

1 always @(*) begin
2     case (address[6:2])
3         5'd0: instruction = 32'b00000000101000000110000010010011;
4         5'd1: instruction = 32'b000000000001100000110000100010011;
5         5'd2: instruction = 32'b01000000001000001000000110110011;
6         5'd3: instruction = 32'b0000000000011000011111001000110011;
7         5'd4: instruction = 32'b000000000001000001101001010110011;
8         5'd5: instruction = 32'b0000000000011000000000000000100011;
9         5'd6: instruction = 32'b0000000000000000000000001100000011;
10        5'd7: instruction = 32'b000000000001100110000010001100011;
11        5'd8: instruction = 32'b00000110001100000110001110010011;
12        5'd9: instruction = 32'b00000000111100000110010000010011;
13        default: instruction = 32'b0;
14    endcase
15 end

```

Como o PC trabalha com endereços em bytes, avançando de 4 em 4, foi utilizado `address[6:2]` para transformar o endereço do PC em índice da memória de instruções. Assim, os endereços 0, 4, 8 e 12 passam a acessar as posições 0, 1, 2 e 3 da memória.

2.10 Caminho de dados

O módulo `data_pathR5` integra todos os blocos do processador. Ele conecta o PC à memória de instruções, envia os campos da instrução para a unidade de controle, banco de

registradores e gerador de imediatos, e define o fluxo dos dados até a ALU e a memória de dados.

Listing 8: Integração dos principais módulos no caminho de dados.

```

1 pc pcDP(pcInDP, pcOutDP, clockDP, resetDP, enableDP);
2 Instruction_mem insMem(pcOutDP, instructionDP);
3
4 Control ctrlDP(
5     instructionDP[6:0], branchDP, memReadDP, memToRegDP,
6     aluOpDP, memWriteDP, aluSrcDP, regWriteDP
7 );
8
9 Reg_mem regMem(
10     clockDP, resetDP, enableDP, regWriteDP,
11     instructionDP[19:15], instructionDP[24:20],
12     instructionDP[11:7], writeRegDataDP,
13     readData1DP, readData2DP,
14     debugRegAddr, debugRegData
15 );
16
17 immGen immGenDP(instructionDP, immGenOutDP);
18 Alu_Control aluControlDP(
19     instructionDP[31:25], instructionDP[14:12],
20     aluOpDP, aluCtrlDP
21 );
22
23 mux2In muxAlu(readData2DP, immGenOutDP, muxAluOutDP, aluSrcDP);
24 alu aluDP(readData1DP, muxAluOutDP, aluCtrlDP, aluOutDP, aluZeroDP);

```

A atualização do PC ocorre por meio de dois caminhos possíveis: o próximo endereço sequencial, obtido por $PC + 4$, ou o endereço de branch, obtido por $PC + \text{imediato}$. O multiplexador final escolhe entre essas opções de acordo com o resultado do *beq*.

Listing 9: Atualização do PC e decisão de branch.

```

1 somador add4(32'd4, pcOutDP, add4OutDP);
2 somador addBranch(pcOutDP, immGenOutDP, addBranchDP);
3
4 assign branchTakenDP = branchDP & aluZeroDP;
5
6 mux2In muxPC(add4OutDP, addBranchDP, pcInDP, branchTakenDP);

```

3 Simulação

Para a simulação foi utilizado um testbench em Verilog com clock automático. A simulação executa o programa gravado na memória de instruções e, ao final, imprime os 32 registradores e as primeiras posições da memória de dados.

3.1 Comando utilizado

A simulação foi executada com o Icarus Verilog no terminal:

Listing 10: Comando utilizado para compilar e executar a simulação.

```
iverilog -g2012 -o grupo22.vvp grupo22_limpo.v  
vvp grupo22.vvp
```

3.2 Código de teste em ASM

Listing 11: Programa de teste em Assembly.

```
ori x1, x0, 10  
ori x2, x0, 3  
sub x3, x1, x2  
and x4, x1, x3  
srl x5, x1, x2  
sb x3, 0(x0)  
lb x6, 0(x0)  
beq x6, x3, 8  
ori x7, x0, 99  
ori x8, x0, 15
```

Esse programa foi escolhido para exercitar todas as instruções do grupo. As duas primeiras instruções inicializam os registradores x1 e x2 com os valores 10 e 3. Em seguida, são testadas as operações *sub*, *and* e *srl*. Depois, *sb* salva o valor de x3 na memória, e *lb* carrega esse valor para x6. Por fim, *beq* compara x6 e x3. Como os dois possuem o valor 7, a instrução que escreveria em x7 é pulada.

3.3 Código em linguagem de máquina

Listing 12: Instruções em binário inseridas no processador.

```
00000000101000000110000010010011  
0000000000011000000110000100010011
```

```

01000000001000001000000110110011
00000000001100001111001000110011
00000000001000001101001010110011
0000000000110000000000000100011
000000000000000000000001100000011
00000000001100110000010001100011
00000110001100000110001110010011
00000000111100000110010000010011

```

3.4 Resultados esperados da simulação

Ao final da execução, os registradores relevantes devem apresentar os seguintes valores:

Registrador	Valor esperado	Justificativa
x1	10	valor carregado por <i>ori</i>
x2	3	valor carregado por <i>ori</i>
x3	7	resultado de <i>sub x3, x1, x2</i>
x4	2	resultado de <i>and x4, x1, x3</i>
x5	1	resultado de <i>srl x5, x1, x2</i>
x6	7	valor carregado da memória por <i>lb</i>
x7	0	instrução pulada pelo <i>beq</i>
x8	15	valor executado após o desvio

Na memória de dados, a posição zero deve conter o valor 7, pois a instrução *sb x3, 0(x0)* armazena o byte menos significativo de x3 nessa posição.

4 Quartus e FPGA

A segunda parte consistiu na adaptação do processador para síntese no Quartus Prime e execução na placa DE2-115. O projeto foi configurado para o dispositivo EP4CE115F29C7, pertencente à família Cyclone IV E.

4.1 Configuração do projeto no Quartus

No Quartus, foi criado um projeto com o módulo principal DE2_115. Esse módulo conecta o processador aos recursos físicos da placa, como botões, switches, LEDs e displays de sete segmentos.

Listing 13: Configurações principais do projeto no Quartus.

```
Family: Cyclone IV E
Device: EP4CE115F29C7
Top-level entity: DE2_115
Arquivo principal: grupo22_fpga.v
Arquivo de restricao de tempo: trabaio.sdc
```

Também foi utilizado um arquivo de pinagem para associar os sinais do módulo DE2_115 aos pinos físicos da placa. A restrição de clock foi definida no arquivo `trabaio.sdc`, considerando o clock de 50 MHz da placa.

Listing 14: Restrição de clock utilizada no arquivo `trabaio.sdc`.

```
create_clock -name CLOCK_50 -period 20.000 [get_ports {CLOCK_50}]
```

4.2 Módulo principal da FPGA

O módulo DE2_115 é responsável por controlar a interação entre a placa e o processador. Nele são definidos os botões utilizados para reset, clock manual e seleção do registrador exibido.

Listing 15: Entradas e saídas principais do módulo DE2_115.

```
1 module DE2_115(
2     input CLOCK_50,
3     input [3:0] KEY,
4     input [17:0] SW,
5     output [17:0] LEDR,
6     output [8:0] LEDG,
7     output [6:0] HEX0,
8     output [6:0] HEX1,
9     output [6:0] HEX2,
10    output [6:0] HEX3,
11    output [6:0] HEX4,
12    output [6:0] HEX5,
13    output [6:0] HEX6,
14    output [6:0] HEX7
15 );
```

A placa utiliza os botões KEY com lógica ativa em nível baixo. Por isso, o reset foi definido a partir da inversão de `KEY[1]`.

Listing 16: Mapeamento dos botões principais.

```

1 assign reset = ~KEY[1];
2
3 button_pulse botaoClock(CLOCK_50, reset, KEY[0], stepPulse);
4 button_pulse botaoReg(CLOCK_50, reset, KEY[2], regPulse);

```

Assim, o KEY0 funciona como clock manual, avançando uma instrução por vez, o KEY1 funciona como reset, e o KEY2 muda o registrador mostrado nos displays e LEDs.

4.3 Exibição do PC e dos registradores

Para atender à parte extra, o projeto permite visualizar tanto o PC quanto um registrador escolhido. O PC é exibido nos displays HEX6 e HEX7. O índice do registrador selecionado é exibido em HEX4 e HEX5. O valor do registrador é exibido em HEX0 até HEX3 e também nos LEDs vermelhos.

Listing 17: Exibição do valor do registrador e do índice selecionado.

```

1 assign LEDR = regValue[17:0];
2 assign LEDG[4:0] = regIndex;
3 assign LEDG[8:5] = 4'b0000;
4
5 seg7 h0(regValue[3:0], HEX0);
6 seg7 h1(regValue[7:4], HEX1);
7 seg7 h2(regValue[11:8], HEX2);
8 seg7 h3(regValue[15:12], HEX3);
9
10 seg7 h4(regOnes, HEX4);
11 seg7 h5(regTens, HEX5);
12
13 seg7 h6(pcOnes, HEX6);
14 seg7 h7(pcTens, HEX7);

```

A navegação entre registradores é feita com KEY2. O switch SW1 define se a navegação será crescente ou decrescente.

Listing 18: Seleção do registrador exibido na FPGA.

```

1 always @(posedge CLOCK_50 or posedge reset) begin
2     if (reset)
3         regIndex <= 5'd0;
4     else if (regPulse) begin
5         if (SW[1])
6             regIndex <= (regIndex == 5'd0) ? 5'd31 : regIndex - 5'd1;
7         else

```



```

8         regIndex <= (regIndex == 5'd31) ? 5'd0 : regIndex + 5'd1;
9     end
10 end

```

4.4 Comandos na placa

A organização dos comandos físicos ficou definida da seguinte forma:

Entrada/Saída	Função
KEY0	Clock manual, executando uma instrução por acionamento.
KEY1	Reset do processador.
KEY2	Alterna o registrador exibido.
SW1	Define a direção da navegação entre registradores.
HEX6 e HEX7	Exibição do PC.
HEX4 e HEX5	Exibição do índice do registrador selecionado.
HEX0 a HEX3	Exibição do valor do registrador selecionado.
LEDG0 a LEDG4	Exibição binária do índice do registrador.
LEDR0 a LEDR17	Exibição binária do valor do registrador.

A execução física foi demonstrada em vídeo, mostrando o reset, o avanço do PC, o salto produzido pelo *beq* e a consulta dos registradores relevantes.

5 Resultados

5.1 Resultado da simulação

A simulação demonstrou que o caminho de dados executou corretamente as instruções do grupo 22. O resultado final esperado foi obtido nos registradores principais: x1 recebeu 10, x2 recebeu 3, x3 recebeu 7, x4 recebeu 2, x5 recebeu 1, x6 recebeu 7, x7 permaneceu zerado e x8 recebeu 15.

O valor de x7 permanecer em zero é importante porque comprova o funcionamento da instrução *beq*. Como x6 e x3 tinham o mesmo valor, o desvio foi tomado e a instrução que atribuiria 99 a x7 foi ignorada. Em seguida, o fluxo do programa continuou na instrução que atribuiu 15 a x8.

5.2 Resultado no Quartus

O projeto foi compilado no Quartus Prime Lite para a placa DE2-115. A compilação completa foi concluída com zero erros, permitindo a geração do arquivo `.sof` utilizado para programação da FPGA. Durante o processo, foram configurados o dispositivo EP4CE115F29C7, o módulo principal DE2_115, o arquivo de pinagem da placa e o arquivo de restrição de clock `trabaoio.sdc`.

5.3 Resultado na FPGA

Na placa, o KEY1 foi utilizado para resetar o processador e o KEY0 para avançar uma instrução por vez. O PC foi exibido nos displays HEX6 e HEX7. Ao executar a instrução *beq*, o PC saltou da instrução correspondente ao desvio para a instrução de destino, pulando a instrução que escreveria em x7.

O KEY2 permitiu navegar entre os registradores, enquanto o SW1 definiu a direção da navegação. Os valores dos registradores puderam ser conferidos nos displays e LEDs, incluindo $x3 = 7$, $x6 = 7$, $x7 = 0$ e $x8 = 15$.

6 Código no GitHub

Os arquivos do projeto foram organizados no GitHub contendo o código Verilog, arquivos de simulação, projeto do Quartus, código Assembly de teste, instruções em binário, estado inicial dos registradores e estado inicial da memória de dados.

7 Código Github

O código-fonte completo do projeto, incluindo os arquivos Verilog, arquivos de entrada, documentação e projeto do Quartus, está disponível em:

<https://github.com/vini-amm/tp02-grupo22-riscv>

8 Considerações finais

A implementação do caminho de dados simplificado do RISC-V para o grupo 22 permitiu verificar o funcionamento integrado de diferentes componentes da arquitetura. O projeto exigiu a implementação de instruções aritméticas, lógicas, de acesso à memória e de desvio condicional.

A simulação foi importante para validar o comportamento do processador antes da síntese. Em seguida, a adaptação para a placa DE2-115 exigiu a criação do módulo principal da FPGA, a configuração dos botões, displays e LEDs, além da definição da pinagem e do arquivo de restrição de clock no Quartus.

Como melhoria, o processador poderia ser expandido para aceitar mais instruções do conjunto RISC-V, além de utilizar uma memória de instruções mais flexível e um mecanismo mais completo de depuração na placa.

Referências

- [1] PATTERSON, David A.; HENNESSY, John L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2017.
- [2] RISC-V Interpreter. Disponível em: <https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/>. Acesso em: jun. 2026.